UNIVERSIDADE FEDERAL DE SÃO PAULO - CAMPUS SÃO JOSÉ DOS CAMPOS

Compilador C-Minus

Gabriel Leal Bonavina - 168.858

Jonas Lucas Durão - 168.893

Marcelo Fukuya Ohno - 168.989

São José dos Campos, SP

2025

Sumário

1 Introdução **5**

1.1 Objetivos do Projeto 5

1.2 Linguagem C-Minus 5

2 Análise Léxica **6**

2.1 Especificação dos Tokens 6

2.2 Tratamento de Comentários e Erros 6

3 Tabela de Símbolos **6**

3.1 Estrutura e Gerenciamento de Escopo 7

3.2 Visualização da Tabela de Símbolos 7

4 Análise Sintática **8**

4.1 Descrição 8

4.2 Gramática da Linguagem 9

4.2.1 Estrutura do Programa 9

4.2.2 Declarações de Variáveis 9

4.2.3 Declarações de Funções 9

4.2.4 Comandos 10

4.2.5 Expressões 11

4.3 Precedência e Associatividade 11

5 Análise Semântica **12**

5.1 Contexto do Parser 12

5.2 Verificações Semânticas 12

5.2.1 Declaração de Variáveis 12

5.2.2 Declaração de Funções 13

5.2.3 Uso de Variáveis 13

5.2.4 Verificação de Tipos 13

5.2.5 Chamadas de Função 15

5.3 Funções Built-in 15

6 Árvore Sintática Abstrata (AST) **15**

6.1 Descrição 15

6.2 Estrutura de Nós 16

6.3 Tipos de Nós 16

6.3.1 Nós de Statement (STMTK) 16

6.3.2 Nós de Expressão (EXPK) 17

6.3.3 Nós de Variável (VARK) 17

6.4 Construção da AST 17
6.4.1 Padrão de Declaração de Variáveis 17
6.4.2 Padrão de Expressões 18
6.4.3 Padrão de Comandos de Controle 20
6.5 Exemplo de AST 21
6.6 Visualização da AST 21
6.6.1 Visualização Textual 21
6.6.2 Visualização Gráfica com GraphViz 22

7 Geração de Código Intermediário 24
7.1 Estrutura do Gerador de Codigo 24
7.1.1 Geradores de Temporários e Labels 25
7.1.2 Geração de Código para Expressões 25
7.1.3 Geração de Código para Comandos 25
7.1.4 Algoritmo de Percurso da Árvore 26
7.2 Formato do Código TAC 26
7.2.1 Implementação da Geração de Expressões 26
7.2.2 Implementação da Geração de Comandos 28
7.3 Exemplo de código TAC gerado 30

8 Tratamento de Erros 32
8.1 Erros Léxicos 32
8.2 Erros Sintáticos 32
8.3 Erros Semânticos 32

9 Arquitetura do Compilador 32
9.1 Fluxo de Compilação 32
9.2 Módulos do Compilador 33
9.2.1 cminusLex.l (Analisador Léxico) 33
9.2.2 cminusSintSem.y (Parser + Semântica + AST) 33
9.2.3 parser_context.h/c (Contexto do Parser) 33
9.2.4 code_generator.h/c (Gerador de Código TAC) 34
9.3 Função Main 34
9.4 Estruturas de Dados Principais 35

10 Exemplos de Uso 36
10.1 Programa Correto 36
10.2 Programa com Erros 40

11 Compilação e Execução **42**
11.1 Ferramentas Necessárias 42
11.2 Processo de Compilação 42
11.3 Makefile 42

12 Conclusão **42**
12.1 Separação do Trabalho 43

13 Referências **43**

1 Introdução

Este documento apresenta a implementação completa de um compilador para a linguagem C-Minus, desenvolvido como trabalho final da disciplina Compiladores. O compilador foi implementado utilizando as ferramentas Flex (analisador léxico) e Bison (analisador sintático), com análise semântica, tabela de símbolos e construção de Árvore Sintática Abstrata (AST) integradas, além de a geração de código intermediário no formato de código de três endereços (TAC).

1.1 Objetivos do Projeto

O projeto tem como objetivos principais:

- Implementar todas as fases de um compilador: análise léxica, sintática, semântica, construção da AST e geração de código intermediário
- Detectar e reportar erros em cada fase da compilação
- Construir uma tabela de símbolos com suporte a escopos aninhados
- Gerar uma representação em árvore do código fonte
- Realizar verificação de tipos e validação semântica
- Gerar um código intermediário no formato TAC ao final da análise

1.2 Linguagem C-Minus

C-Minus é um subconjunto da linguagem C projetado para fins didáticos. A linguagem possui as seguintes características:

- Dois tipos de dados: `int` e `void`
- Suporte a arrays unidimensionais
- Estruturas de controle: `if-else` e `while`
- Funções com parâmetros e retorno
- Expressões aritméticas e relacionais
- Escopo de variáveis com regras de visibilidade

2 Análise Léxica

A análise léxica constitui a primeira fase do compilador, encarregada de varrer o código-fonte e agrupar os caracteres em unidades lógicas denominadas tokens. Esta etapa foi implementada utilizando a ferramenta **Flex** no arquivo `cminusLex.l`.

2.1 Especificação dos Tokens

O analisador foi configurado para identificar as construções fundamentais da linguagem C-Minus. A categorização dos tokens reconhecidos compreende:

Palavras Reservadas: `else`, `if`, `int`, `return`, `void`, `while`.

Operadores: Incluem os relacionais (`<`, `<=`, `>`, `>=`, `==`, `!=`), aritméticos (`+`, `-`, `*`, `/`) e de atribuição (`=`).

Delimitadores: Símbolos de pontuação e escopo, como parênteses `()`, colchetes `[]`, chaves {}, ponto e vírgula `;` e vírgula `,`.

Identificadores e Literais: O token **ID** representa variáveis e funções (iniciados por letra), enquanto **NUM** representa literais inteiros.

2.2 Tratamento de Comentários e Erros

O analisador suporta comentários de linha única (iniciados por `//`) e de múltiplas linhas (delimitados por `/* ... */`). Para os comentários em bloco, utilizou-se o recurso de *start conditions* do Flex (estado `COMMENT`), assegurando o tratamento correto de quebras de linha e detectando erros de fechamento.

Erros léxicos, como caracteres inválidos, disparam mensagens informativas apontando a linha da ocorrência, conforme o exemplo:

```
ERRO LEXICO:    caractere nao reconhecido: '@' - LINHA: 5
```

3 Tabela de Símbolos

A tabela de símbolos é a estrutura central para a análise semântica e criada na análise léxica, sendo responsável por armazenar metadados de identificadores, como tipo, escopo e natureza (variável, função ou vetor).

3.1 Estrutura e Gerenciamento de Escopo

A implementação utiliza uma lista encadeada de símbolos (`struct Simbolo`), onde cada nó contém informações cruciais como nome, tipo de dado, número de parâmetros (para funções) e linha de declaração.

Para suportar o aninhamento de blocos da linguagem C-Minus, implementou-se um sistema de escopos baseado em pilha (`struct Escopo`). As operações principais incluem `enter_scope()` e `leave_scope()` para navegação na hierarquia, e funções de *lookup* que buscam identificadores tanto no escopo atual quanto nos ancestrais, garantindo as regras de visibilidade da linguagem.

As estruturas principais estão definidas abaixo:

```c
typedef struct Simbolo {
    char *nome;
    TipoVar tipo;        /* int, void */
    TipoSimbolo kind;    /* VAR , ARRAY , FUNC */
    int tamanho;         /* para arrays */
    int num_params;      /* para funcoes (mantido por compatibilidade) */
    TipoVar *param_types; /* tipos dos parametros */
    int linha;
    int is_param;            /* 1 se este simbolo e um parametro */
    struct Escopo *def_scope; /* escopo associado (para funcoes: o escopo do corpo) */
    struct Simbolo *prox;
} Simbolo;

typedef struct Escopo {
    Simbolo *simbolos;    /* Lista de simbolos deste escopo */
    struct Escopo *pai;   /* Referencia ao escopo superior */
} Escopo;
```

Listing 1: Estruturas de dados para Símbolos e Escopos

3.2 Visualização da Tabela de Símbolos

A tabela armazena sete atributos principais para cada símbolo. O atributo **Escopo** indica a profundidade da declaração (0 para global, incrementado para escopos internos). Considere o seguinte trecho de código para validação:

```c
int gcd(int u, int v) {
    if (v == 0) return u;
```

```
    else return gcd(v, u - (u/v) * v);
}
void main(void) {
    int x; int y;
    x = input();
    y = input();
    output(gcd(x, y));
}
```

Após a análise, o estado final da tabela de símbolos é apresentado na Tabela 1. Note que as variáveis `u` e `v` pertencem ao escopo da função `gcd` (Escopo 1), enquanto `x` e `y` pertencem à `main` (Escopo 2).

Tabela 1: Estado da Tabela de Símbolos após análise

Nome	**Tipo**	**Categ.**	**Escopo**	**Tam.**	**Params**	**Linha**
main	void	FUNC	0	-	0	8
gcd	int	FUNC	0	-	2	1
output	void	FUNC	0	-	0	0
input	int	FUNC	0	-	0	0
v	int	VAR	1	-	-	1
u	int	VAR	1	-	-	1
y	int	VAR	2	-	-	9
x	int	VAR	2	-	-	9

4 Análise Sintática

4.1 Descrição

A análise sintática verifica se a sequência de tokens produzida pelo analisador léxico está de acordo com a gramática da linguagem C-Minus. Foi implementada utilizando o gerador de parsers Bison no arquivo `cminusSintSem.y`.

O parser utiliza uma estrutura `ParserContext` para gerenciar o estado da compilação, permitindo passagem de contexto entre as ações semânticas:

```
%parse-param {ParserContext *ctx}

void yyerror(ParserContext *ctx, const char *s);
```

Listing 2: Declaracao do parser com contexto

Esta abordagem permite gerenciar escopos através do contexto compartilhado, acumulando erros sem interromper a compilação e mantendo o estado consistente entre diferentes

regras gramaticais. Adicionalmente, o contexto armazena a raiz da AST construída, facilitando o acesso posterior para visualização e geração de código.

4.2 Gramática da Linguagem

A gramática de C-Minus segue as seguintes produções principais:

4.2.1 Estrutura do Programa

```
program -> declaration-list

declaration-list -> declaration-list declaration
                  | declaration

declaration -> var-declaration
             | fun-declaration
```

Listing 3: Produção do programa

4.2.2 Declarações de Variáveis

```
var-declaration -> type-specifier ID ;
                 | type-specifier ID [ NUM ] ;

type-specifier -> int | void
```

Listing 4: Declaração de variáveis

As variáveis podem ser simples ou arrays. Arrays devem ter tamanho especificado na declaração.

4.2.3 Declarações de Funções

```
fun-declaration -> type-specifier ID ( params ) compound-stmt

params -> param-list | void | epsilon

param-list -> param-list , param
            | param

param -> type-specifier ID
       | type-specifier ID [ ]
```

Listing 5: Declaração de funções

Funções podem ter tipo de retorno `int` ou `void`. Parâmetros podem ser variáveis simples ou arrays (sem especificar tamanho).

Implementação: A declaração de função utiliza acoes intermediarias para gerenciar escopos:

```
fun_declaration :
    type_specifier ID
    {
        insert_function_ctx(ctx, $2, $1, yylineno);
        Simbolo *func_sym = lookup_symbol_current_ctx(ctx, $2)
            ;
        enter_scope_ctx(ctx);  /* Novo escopo para parametros
            */
        if (func_sym) {
            func_sym->def_scope = ctx->escopo_atual;
        }
    }
    LPAREN params RPAREN compound_stmt
    {
        leave_scope_ctx(ctx);  /* Sai do escopo da funcao */
        /* Construcao da AST... */
    }
    ;
```

Listing 6: Regra de declaracao de funcao

Esta estrutura implementa a semântica de escopo correta para funções: primeiro, a função é declarada no escopo pai, tornando-a visível para chamadas externas. Em seguida, um novo escopo filho é criado especificamente para os parâmetros e o corpo da função, garantindo o isolamento de variáveis locais. Finalmente, ao término do processamento do corpo, o escopo é fechado com `leave_scope_ctx()`, retornando ao escopo original e mantendo a hierarquia correta.

4.2.4 Comandos

```
statement -> expression-stmt
          | compound-stmt
          | selection-stmt
          | iteration-stmt
          | return-stmt

compound-stmt -> { local-declarations statement-list }

```

```
selection-stmt -> if ( expression ) statement
                | if ( expression ) statement else statement

iteration-stmt -> while ( expression ) statement

return-stmt -> return ;
            | return expression ;
```

Listing 7: Estrutura de comandos

4.2.5 Expressões

```
expression -> var = expression
           | simple-expression

simple-expression -> additive-expression relop additive-
   expression
                  | additive-expression

additive-expression -> additive-expression addop term
                    | term

term -> term mulop factor
     | factor

factor -> ( expression )
       | var
       | call
       | NUM

var -> ID
    | ID [ expression ]

call -> ID ( args )
```

Listing 8: Estrutura de expressões

4.3 Precedência e Associatividade

As regras de precedência foram configuradas no Bison:

```
%left PLUS MINUS        /* Menor precedencia */
%left TIMES DIVIDE
```

```
%right ASSIGN
%left LT LE GT GE EQ NE /* Maior precedencia */
```

5 Análise Semântica

5.1 Contexto do Parser

A análise semântica é integrada ao parser através de uma estrutura de contexto definida em `parser_context.h`:

```
typedef struct ParserContext {
    Escopo *escopo_atual;       /* Escopo em processamento */
    Escopo *lista_escopos;      /* Lista de todos os escopos */
    TreeNode *ast_root;         /* Raiz da AST */
    int has_errors;             /* Flag de erros */
} ParserContext;
```

Listing 9: Estrutura ParserContext

Esta estrutura centraliza o estado da compilação e é passada como parâmetro para todas as funções semânticas. O design permite acesso consistente à tabela de símbolos através do campo `escopo_atual`, enquanto o rastreamento de erros via `has_errors` possibilita a continuidade da análise mesmo na presença de problemas. A construção progressiva da AST ocorre de forma integrada, com o campo `ast_root` mantendo a referência para a estrutura completa. Este isolamento de estado viabiliza múltiplas compilações independentes sem interferência de contextos anteriores.

5.2 Verificações Semânticas

5.2.1 Declaração de Variáveis

- Variáveis não podem ser declaradas mais de uma vez no mesmo escopo
- Arrays não podem ter tipo `void`
- Tamanho de arrays deve ser especificado na declaração

```
int x;
int x;  // ERRO: variavel 'x' ja declarada
void arr[10];  // ERRO: array nao pode ser void
```

Listing 10: Exemplo de verificação

5.2.2 Declaração de Funções

- Funções não podem ser declaradas mais de uma vez
- Parâmetros não podem ter tipo `void`
- Função `main` deve existir

5.2.3 Uso de Variáveis

- Variáveis devem ser declaradas antes do uso
- Operação de indexação só pode ser aplicada a arrays
- Índices de arrays devem ser do tipo `int`

```
y = x + 5;   // ERRO se 'x' nao foi declarado
int a;
a[0] = 10;   // ERRO: 'a' nao e um array
int b[10];
b[x] = 5;    // ERRO se 'x' nao for int
```

Listing 11: Exemplo de verificação de uso

5.2.4 Verificação de Tipos

O compilador realiza verificação de tipos em expressões:

- Operações aritméticas requerem operandos do tipo `int`
- Operações relacionais requerem operandos do tipo `int`
- Tipo `void` não pode ser usado em expressões
- Atribuições devem ter tipos compatíveis
- Não é possível atribuir a um array completo

Implementacao: A função `check_expression_type_ctx` valida operações:

```
TipoVar check_expression_type_ctx(ParserContext *ctx,
                                  const char *op,
                                  TipoVar t1, TipoVar t2,
                                  int linha) {
    if (t1 == TYPE_ERROR || t2 == TYPE_ERROR) {
        return TYPE_ERROR;
    }
```

```

    if (t1 == TYPE_VOID || t2 == TYPE_VOID) {
        fprintf(stderr, "ERRO SEMANTICO: linha %d: "
                "operacao %s com tipo void\n", linha, op);
        ctx->has_errors = 1;
        return TYPE_ERROR;
    }

    if (t1 != TYPE_INT || t2 != TYPE_INT) {
        fprintf(stderr, "ERRO SEMANTICO: linha %d: "
                "operacao %s requer operandos inteiros\n",
                linha, op);
        ctx->has_errors = 1;
        return TYPE_ERROR;
    }

    return TYPE_INT;
}
```

Listing 12: Verificacao de tipos em expressoes

Esta função e chamada nas regras de expressões aritméticas e relacionais:

```
additive_expression :
    additive_expression addop term
    {
        TipoVar t1 = $1 ? $1->type : TYPE_ERROR;
        TipoVar t2 = $3 ? $3->type : TYPE_ERROR;
        TipoVar result = check_expression_type_ctx(ctx,
                                "aditivo", t1, t2,
                                   yylineno);

        // Construcao da AST
        $$ = newExpNode(OPK);
        $$->child[0] = $1;
        $$->child[1] = $3;
        $$->op = $2;
        $$->type = result;
        $$->lineno = yylineno;
    }
    | term { $$ = $1; }
    ;
```

Listing 13: Uso na regra additive_expression

```
void f() { }
int x;
x = f();  // ERRO: tipos incompativeis

int arr[10];
arr = 5;  // ERRO: nao pode atribuir a array completo

if (f()) { }  // ERRO: condicao deve ser int
```

Listing 14: Exemplos de verificação de tipos

5.2.5 Chamadas de Função

- Função deve estar declarada
- Identificador chamado deve ser uma função

5.3 Funções Built-in

O compilador pré-declara duas funções built-in no escopo global:

- `int input(void)`: Lê um valor inteiro da entrada
- `void output(int)`: Imprime um valor inteiro na saída

6 Árvore Sintática Abstrata (AST)

6.1 Descrição

A Árvore Sintática Abstrata (AST) é uma representação hierárquica do código fonte que elimina detalhes sintáticos irrelevantes e mantém apenas a estrutura semântica do programa.

A AST e construida simultaneamente com a analise sintatica e semantica, sendo armazenada no campo `ast_root` do contexto do parser:

```
program :
    declaration_list
    {
        $$ = $1;
        ctx->ast_root = $$;  /* Armazena raiz da AST */
        printf("Analise sintatica concluida com sucesso!\n");
        if (!ctx->has_errors) {
```

```
            printf("Analise semantica: Nenhum erro encontrado.\n");
        }
    }
    ;
```

Listing 15: Armazenamento da AST no contexto

6.2 Estrutura de Nós

A AST é composta por nós representando diferentes elementos da linguagem:

```
typedef struct treeNode {
    struct treeNode *child[MAXCHILDREN];
    struct treeNode *sibling;
    int lineno;
    NodeKind nodekind;  /* STMTK, EXPK, VARK */
    union {
        StmtKind stmt;
        ExpKind exp;
        struct Identifier var;
    } kind;
    int op;             /* Token do operador */
    TipoVar type;       /* Tipo de dado */
} TreeNode;
```

Listing 16: Estrutura de nó da AST

6.3 Tipos de Nós

6.3.1 Nós de Statement (STMTK)

Representam comandos e declarações de tipos:

- **INTEGERK**: Especificador de tipo `int`
- **VOIDK**: Especificador de tipo `void`
- **IFK**: Comando `if-else`
- **WHILEK**: Comando `while`
- **RETURNK**: Comando `return`
- **COMPK**: Bloco composto {...}

6.3.2 Nós de Expressão (EXPK)

Representam expressões e operações:

- **OPK**: Operadores (aritméticos, relacionais)
- **CONSTK**: Constantes numéricas
- **IDK**: Identificadores simples
- **ASSIGNK**: Operação de atribuição
- **CALLK**: Chamada de função
- **VECTORK**: Acesso a array

6.3.3 Nós de Variável (VARK)

Representam identificadores de variáveis, funções e arrays:

- **DECLK**: Uso em declaração
- **ACCESSK**: Uso em acesso/referência

6.4 Construção da AST

A AST é construída durante a análise sintática seguindo o padrão:

1. **Declarações**: Nó de tipo (int/void) como raiz, com ID como filho
2. **Funções**: Tipo → ID função → [parâmetros, corpo]
3. **Arrays**: ID array com tamanho como filho
4. **Expressões**: Operadores com operandos como filhos
5. **Comandos**: Estrutura com condição e corpo como filhos

6.4.1 Padrão de Declaração de Variáveis

Cada declarção segue o padrão: tipo-especificador contem identificador como filho:

```
var_declaration :
    type_specifier ID SEMI
    {
        /* Analise semantica */
        insert_symbol_ctx(ctx, $2, $1, KIND_VAR, yylineno);

```

```
        /* Construcao da AST - padrao correto */
        $$ = newStmtNode($1 == TYPE_INT ? INTEGERK : VOIDK);
        $$->type = $1;

        /* Criar no ID e anexar como filho */
        $$->child[0] = newVarNode(IDK);
        $$->child[0]->kind.var.attr.name = strdup($2);
        $$->child[0]->kind.var.varKind = KIND_VAR;
        $$->child[0]->kind.var.acesso = DECLK;
        $$->child[0]->type = $1;
        $$->child[0]->lineno = yylineno;

        free($2);
    }
```

Listing 17: Construcao de no de declaracao de variavel

Para arrays, o tamanho e adicionado como neto (filho do ID):

```
| type_specifier ID LBRACK NUM RBRACK SEMI
{
    /* Analise semantica */
    insert_array_ctx(ctx, $2, $4, yylineno);

    /* Tipo como raiz */
    $$ = newStmtNode($1 == TYPE_INT ? INTEGERK : VOIDK);
    $$->type = $1;

    /* ID array como filho */
    $$->child[0] = newVarNode(VECTORK);
    $$->child[0]->kind.var.attr.name = strdup($2);
    $$->child[0]->kind.var.varKind = KIND_ARRAY;

    /* Tamanho como filho do ID array (neto do tipo) */
    $$->child[0]->child[0] = newExpNode(CONSTK);
    $$->child[0]->child[0]->kind.var.attr.val = $4;
    $$->child[0]->child[0]->type = TYPE_INT;
}
```

Listing 18: Construcao de no de array

6.4.2 Padrão de Expressões

Expressoes aritmeticas e relacionais criam nos OPK com operandos como filhos:

```
additive_expression :
    additive_expression addop term
    {
        /* Verifica tipos */
        TipoVar t1 = $1 ? $1->type : TYPE_ERROR;
        TipoVar t2 = $3 ? $3->type : TYPE_ERROR;
        TipoVar result = check_expression_type_ctx(ctx,
                                "aditivo", t1, t2,
                                   yylineno);

        /* Cria no operador */
        $$ = newExpNode(OPK);
        $$->child[0] = $1;  /* Operando esquerdo */
        $$->child[1] = $3;  /* Operando direito */
        $$->op = $2;        /* Token do operador (+, -, etc)
           */
        $$->type = result;
        $$->lineno = yylineno;
    }
```

Listing 19: Construcao de no de operacao

Atribuicoes usam no ASSIGNK:

```
expression :
\begin{figure}
        \centering
        \includesvg[width=1\linewidth]{ast}
        \caption{Enter Caption}
        \label{fig:placeholder}
    \end{figure}
        var ASSIGN expression
    {
        /* Verificacoes semanticas de tipo */
        TipoVar var_type = $1 ? $1->type : TYPE_ERROR;
        TipoVar exp_type = $3 ? $3->type : TYPE_ERROR;

        /* Valida que nao esta atribuindo a array completo */
        if ($1 && $1->nodekind == VARK &&
            $1->kind.var.varKind == KIND_ARRAY &&
            $1->child[0] == NULL) {
            fprintf(stderr, "ERRO: nao pode atribuir a "
                    "array completo\n");
```

```
            ctx->has_errors = 1;
        }

        /* Cria no de atribuicao */
        $$ = newExpNode(ASSIGNK);
        $$->child[0] = $1;  /* Variavel (lado esquerdo) */
        $$->child[1] = $3;  /* Expressao (lado direito) */
        $$->type = var_type;
        $$->lineno = yylineno;
    }
```

Listing 20: Construcao de no de atribuicao

6.4.3 Padrão de Comandos de Controle

Comandos de controle (if, while) tem condicao e corpo como filhos:

```
selection_stmt :
    IF LPAREN expression RPAREN statement
    {
        /* Valida que condicao e inteira */
        if ($3 && $3->type != TYPE_INT) {
            fprintf(stderr, "ERRO: condicao do IF deve "
                    "ser inteira\n");
            ctx->has_errors = 1;
        }

        $$ = newStmtNode(IFK);
        $$->child[0] = $3;    /* Condicao */
        $$->child[1] = $5;    /* Then */
        $$->child[2] = NULL;  /* Else (opcional) */
        $$->lineno = yylineno;
    }
    | IF LPAREN expression RPAREN statement ELSE statement
    {
        /* Versao com else */
        $$ = newStmtNode(IFK);
        $$->child[0] = $3;  /* Condicao */
        $$->child[1] = $5;  /* Then */
        $$->child[2] = $7;  /* Else */
    }
```

Listing 21: Construcao de no IF

6.5 Exemplo de AST

Para o código:

```
int x[10];
int minloc(int a[], int low, int high) {
    int i;
    i = low;
    while (i < high) {
        i = i + 1;
    }
    return i;
}
```

Listing 22: Código fonte exemplo

A AST gerada é:

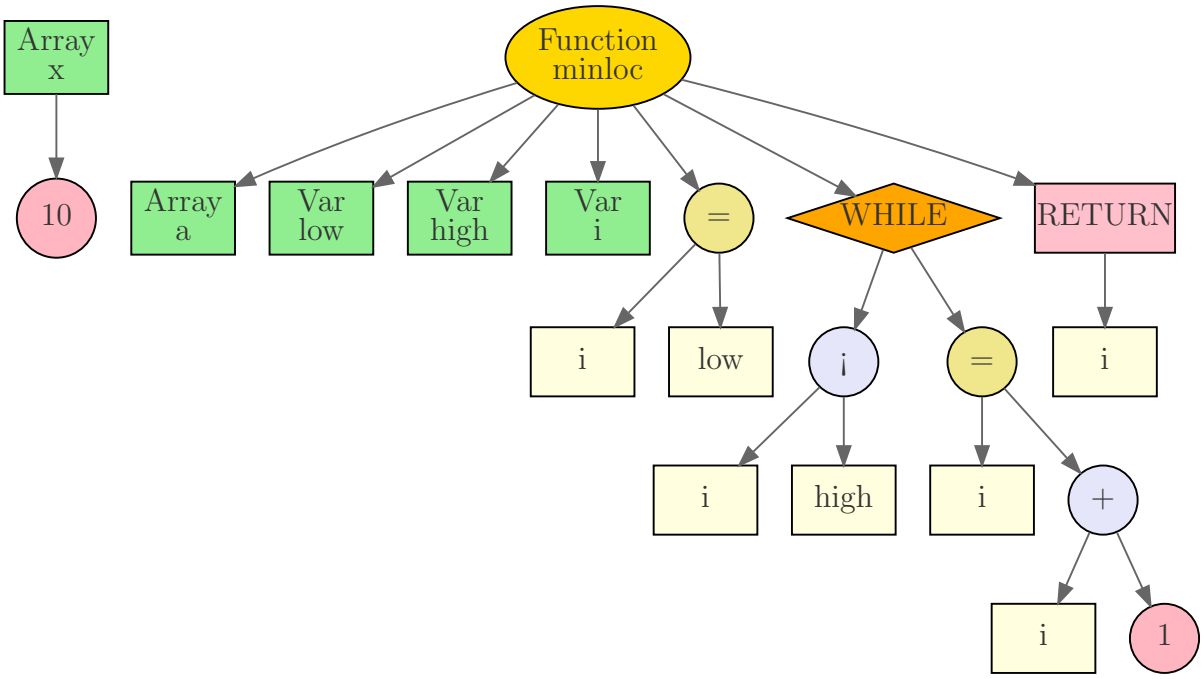

Figura 1: Visualização da árvore do exemplo em Graphviz

6.6 Visualização da AST

O compilador oferece duas formas de visualização da AST:

6.6.1 Visualização Textual

A função `printTree()` percorre a AST em pre-ordem e imprime cada nó com indentação proporcional a profundidade:

```c
void printTree(TreeNode *tree, int indent) {
    if (tree == NULL) return;

    /* Imprime indentacao */
    for (int i = 0; i < indent; i++)
        printf("  ");

    /* Imprime informacao do no baseado no tipo */
    if (tree->nodekind == STMTK) {
        /* Imprime statements (INTEGERK, VOIDK, IFK, etc) */
    } else if (tree->nodekind == VARK) {
        /* Imprime identificadores com contexto */
        printf("[%s]", tree->kind.var.attr.name);
        if (tree->kind.var.acesso == DECLK)
            printf(" (declaration)");
        else
            printf(" (access)");
    } else if (tree->nodekind == EXPK) {
        /* Imprime expressoes (operadores, constantes, etc) */
    }

    /* Recursao para filhos e irmaos */
    for (int i = 0; i < MAXCHILDREN; i++)
        printTree(tree->child[i], indent + 1);
    printTree(tree->sibling, indent);
}
```

Listing 23: Impressao textual da AST

6.6.2 Visualização Gráfica com GraphViz

Além da impressão textual, o compilador gera arquivo DOT para visualização gráfica:

```c
void printTreeDOT(TreeNode *tree, const char *filename) {
    FILE *fp = fopen(filename, "w");
    DotContext ctx = {0, fp};

    /* Cabecalho DOT */
    fprintf(fp, "digraph AST {\n");
    fprintf(fp, "  rankdir=TB;\n");
    fprintf(fp, "  node [fontname=\"Arial\"];\n");

```

```
    /* Gera arvore recursivamente */
    printTreeDOT_simplified(&ctx, tree, -1);

    fprintf(fp, "}\n");
    fclose(fp);
}
```

Listing 24: Geracao de arquivo DOT

A função recursiva `printTreeDOT_simplified` cria nos e arestas no formato DOT:

```
static int printTreeDOT_simplified(DotContext *ctx,
                                   TreeNode *tree,
                                   int parent_id) {
    if (tree == NULL) return -1;

    int current_id = ctx->node_counter++;

    /* Define aparencia do no (cor, forma, label) */
    const char *shape = "box";
    const char *color = "lightblue";
    char label[256];

    /* Gera label baseado no tipo de no */
    if (tree->nodekind == STMTK) {
        /* Statements: [int], [void], If, While, etc */
    } else if (tree->nodekind == VARK) {
        /* Identificadores: [nome] com categoria */
    } else if (tree->nodekind == EXPK) {
        /* Expressoes: Op, Const, Call, etc */
    }

    /* Cria no no arquivo DOT */
    fprintf(fp, "  node%d [label=\"%s\", shape=%s, "
                "fillcolor=%s];\n",
            current_id, label, shape, color);

    /* Liga ao pai */
    if (parent_id >= 0) {
        fprintf(fp, "  node%d -> node%d;\n",
                parent_id, current_id);
    }

```

```
    /* Processa filhos e irmaos recursivamente */
    for (int i = 0; i < MAXCHILDREN; i++) {
        if (tree->child[i]) {
            printTreeDOT_simplified(ctx, tree->child[i],
                                    current_id);
        }
    }

    if (tree->sibling) {
        printTreeDOT_simplified(ctx, tree->sibling,
                                parent_id);
    }

    return current_id;
}
```

Listing 25: Geracao de nos DOT

O arquivo DOT gerado pode ser convertido em imagem:

```
# PNG
dot -Tpng ast.dot -o ast.png

# SVG (recomendado para qualidade)
dot -Tsvg ast.dot -o ast.svg

# PDF
dot -Tpdf ast.dot -o ast.pdf
```

7 Geração de Código Intermediário

A fase de geração de código intermediário é responsável por transformar a árvore sintática abstrata (AST) produzida pela análise sintática em uma representação intermediária de baixo nível, conhecida como código de três endereços (Three-Address Code - TAC). Esta representação facilita a análise e otimização do código, servindo como ponte entre a linguagem de alto nível (C-Minus) e o código de máquina.

7.1 Estrutura do Gerador de Codigo

A implementação do gerador de código intermediário está no módulo `code_generator.c` com interface em `code_generator.h`, e é organizada em funções principais:

7.1.1 Geradores de Temporários e Labels

O gerador utiliza dois contadores globais estáticos para criar nomes únicos:

- `contadorTemporarios`: Contador para variáveis temporárias
- `contadorLabels`: Contador para rótulos

As funções auxiliares `novoTemporario()` e `novoLabel()` utilizam os contadores globais para gerar identificadores únicos e sequenciais. A primeira produz nomes de variáveis temporárias no formato t0, t1, t2, garantindo que cada resultado intermediário tenha um local de armazenamento exclusivo. A segunda gera rótulos no formato L0, L1, L2, que servem como alvos para instruções de desvio no código intermediário. Este mecanismo garante ausência de conflitos de nomes durante todo o processo de geração, independentemente da complexidade do código fonte.

O módulo define três funções estáticas auxiliares que formam o núcleo do gerador. A função `gerarExpressao()` é responsável por processar nós de expressão da AST, calculando valores e retornando os temporários correspondentes. A função `gerarComando()` processa comandos e declarações, incluindo estruturas de controle, declarações de funções e blocos compostos. Complementando essas duas, `gerarComandoExpressao()` trata especificamente de expressões utilizadas como comandos standalone, como atribuições e chamadas de função sem uso do valor de retorno.

7.1.2 Geração de Código para Expressões

A função `gerarExpressao()` processa nós de expressão da AST através de um switch que analisa o tipo do nó e retorna o nome do temporário contendo o resultado. Para constantes numéricas (**CONSTK**), gera uma instrução de atribuição direta no formato `t0 = valor`. Identificadores simples (**IDK**) retornam o próprio nome da variável sem necessidade de temporário intermediário. Operações aritméticas e relacionais (**OPK**) processam recursivamente os operandos esquerdo e direito, gerando código no formato `t0 = esq op dir`. Chamadas de função (**CALLK**) requerem processamento sequencial dos argumentos com instruções `param`, seguidas pela instrução de chamada `t0 = call func, n`. Para acessos a variáveis ou arrays (**VARK**), arrays indexados geram instruções específicas de acesso `t0 = arr[indice]`, enquanto variáveis simples retornam diretamente o nome.

7.1.3 Geração de Código para Comandos

A função `gerarComando()` trata estruturas de controle e declarações, como `IF`, `WHILE` e `RETURN`, além de tratar os tipos `INT` e `VOID`. Também processa declarações de funções (`func ... endfunc`) e arrays.

7.1.4 Algoritmo de Percurso da Árvore

A função `gerarComandoExpressao()` trata especificamente de expressões usadas como comandos, principalmente atribuições (`ASSIGNK`) e chamadas de função (`CALLK`) quando usadas como statements.

A função `percorrerArvore()` implementa um percurso em profundidade da AST, processando cada nó e seus irmãos sequencialmente. O ponto de entrada principal é `codeGen()`, que inicializa o processo imprimindo um cabeçalho e chamando `percorrerArvore()`.

7.2 Formato do Código TAC

O código de três endereços gerado segue o formato:

- **Atribuição**: `temp = valor` ou `var = expressao`
- **Operações**: `t0 = x op y` onde op pode ser +, -, *, /, ¡, ¡=, ¿, ¿=, ==, !=
- **Chamadas**: `t0 = call funcao, numArgs` ou `call funcao, numArgs`
- **Parâmetros**: `param valor`
- **Funções**: `func nome:` e `endfunc`
- **Labels**: `L0:`, `L1:`, etc.
- **Desvios condicionais**: `if_false cond goto label`
- **Desvios incondicionais**: `goto label`
- **Retorno**: `return` ou `return valor`
- **Arrays**: `array nome[tamanho]`, `t0 = arr[indice]`, `arr[indice] = valor`

7.2.1 Implementação da Geração de Expressões

```
static char* gerarExpressao(TreeNode* no) {
    char* temp;
    char* esquerda;
    char* direita;
    char* operador;

    if (no == NULL)
        return NULL;

    if (no->nodekind == EXPK) {
        switch (no->kind.exp) {
```

```
12              case CONSTK:
13                  temp = novoTemporario();
14                  printf("%s = %d\n", temp, no->kind.var.attr.
                       val);
15                  return temp;
16
17              case IDK:
18                  return no->kind.var.attr.name;
19
20              case OPK:
21                  esquerda = gerarExpressao(no->child[0]);
22                  direita = gerarExpressao(no->child[1]);
23                  temp = novoTemporario();
24
25                  switch (no->op) {
26                      case PLUS:     operador = "+"; break;
27                      case MINUS:    operador = "-"; break;
28                      case TIMES:    operador = "*"; break;
29                      case DIVIDE:   operador = "/"; break;
30                      case LT:       operador = "<"; break;
31                      case LE:       operador = "<="; break;
32                      case GT:       operador = ">"; break;
33                      case GE:       operador = ">="; break;
34                      case EQ:       operador = "=="; break;
35                      case NE:       operador = "!="; break;
36                      default:       operador = "?"; break;
37                  }
38
39                  printf("%s = %s %s %s\n", temp, esquerda,
40                         operador, direita);
41                  return temp;
42
43              case CALLK:
44                  {
45                      TreeNode* argumento = no->child[0];
46                      int numArgumentos = 0;
47
48                      /* Processa argumentos */
49                      while (argumento != NULL) {
50                          char* tempArg = gerarExpressao(
                               argumento);
```

```
                        printf("param %s\n", tempArg);
                        numArgumentos++;
                        argumento = argumento->sibling;
                    }

                    temp = novoTemporario();
                    printf("%s = call %s, %d\n", temp,
                           no->kind.var.attr.name,
                              numArgumentos);
                    return temp;
                }

            default:
                return NULL;
        }
    } else if (no->nodekind == VARK) {
        /* Acesso a variavel ou array */
        if (no->kind.var.varKind == KIND_ARRAY &&
            no->child[0] != NULL) {
            /* Acesso a array com indice */
            char* indice = gerarExpressao(no->child[0]);
            temp = novoTemporario();
            printf("%s = %s[%s]\n", temp,
                   no->kind.var.attr.name, indice);
            return temp;
        } else {
            /* Variavel simples */
            return no->kind.var.attr.name;
        }
    }

    return NULL;
}
```

Listing 26: Funcao gerarExpressao

7.2.2 Implementação da Geração de Comandos

```
static void gerarComando(TreeNode* no) {
    char* teste;
    char* labelFalso;
    char* labelFim;
```

```
    char* labelInicio;
    char* valor;

    if (no == NULL)
        return;

    if (no->nodekind == STMTK) {
        switch (no->kind.stmt) {
            case IFK: /* if ou if-else */
                teste = gerarExpressao(no->child[0]);
                labelFalso = novoLabel();
                labelFim = novoLabel();

                printf("if_false %s goto %s\n", teste,
                   labelFalso);
                gerarComando(no->child[1]);

                if (no->child[2] != NULL) {
                    /* Tem else */
                    printf("goto %s\n", labelFim);
                    printf("%s:\n", labelFalso);
                    gerarComando(no->child[2]);
                    printf("%s:\n", labelFim);
                } else {
                    /* Sem else */
                    printf("%s:\n", labelFalso);
                }
                break;

            case WHILEK: /* Loop while */
                labelInicio = novoLabel();
                labelFim = novoLabel();

                printf("%s:\n", labelInicio);
                teste = gerarExpressao(no->child[0]);
                printf("if_false %s goto %s\n", teste,
                   labelFim);
                gerarComando(no->child[1]);
                printf("goto %s\n", labelInicio);
                printf("%s:\n", labelFim);
                break;
```

```
44
45                  case RETURNK: /* Return */
46                      if (no->child[0] != NULL) {
47                          valor = gerarExpressao(no->child[0]);
48                          printf("return %s\n", valor);
49                      } else {
50                          printf("return\n");
51                      }
52                      break;
53
54                  /* ... outros casos ... */
55              }
56          }
57      }
```

Listing 27: Funcao gerarComando - estruturas de controle

7.3 Exemplo de código TAC gerado

Considere o código a seguir:

```
1  int gcd(int u, int v) {
2      if (v == 0)
3          return u;
4      else
5          return gcd(v, u - (u/v) * v);
6  }
7
8  void main(void) {
9      int x; int y;
10     x = input();
11     y = input();
12     output(gcd(x, y));
13 }
```

Listing 28: Exemplo: funcao gcd

Após passar pelo gerador de código intermediário, a saída gerada é:

```
1  *** CODIGO INTERMEDIARIO (3 ENDERECOS) ***
2
3  func gcd:
4      param u
5      param v
```

```
    t0 = 0
    t1 = v == t0
    if_false t1 goto L0
    return u
    goto L1
    L0:
    param v
    t2 = u / v
    t3 = t2 * v
    t4 = u - t3
    param t4
    t5 = call gcd, 2
    return t5
    L1:
endfunc

func main:
    t6 = call input, 0
    x = t6
    t7 = call input, 0
    y = t7
    param x
    param y
    t8 = call gcd, 2
    param t8
    call output, 1
endfunc

****************************************
```

Listing 29: Codigo TAC gerado

Este código intermediário demonstra a estrutura completa do formato TAC gerado. As declarações de funções são delimitadas por `func nome:` e `endfunc`, criando blocos bem definidos. Os parâmetros aparecem de duas formas: `param u`, `param v` declaram parâmetros formais na definição da função, enquanto nas chamadas `param x`, `param y` preparam os argumentos. A sequência de temporários t0 a t8 ilustra o uso sistemático de variáveis intermediárias para armazenar resultados parciais, como nas operações aritméticas `t2 = u / v`, `t3 = t2 * v`, `t4 = u - t3`. Comparações como `t1 = v == t0` geram valores booleanos usados em desvios condicionais `if_false t1 goto L0`, onde labels L0 e L1 marcam pontos de destino no fluxo de controle. A recursividade aparece naturalmente em `t5 = call gcd, 2`, e os retornos podem carregar valores (`return t5`) ou serem vazios

(`return`).

A estrutura linearizada e simplificada do código TAC facilita análises posteriores de fluxo de controle e dependências de dados. Esta representação intermediária serve como base sólida tanto para otimizações quanto para geração de código de máquina para diferentes arquiteturas.

8 Tratamento de Erros

O compilador detecta e reporta diversos tipos de erros em todas as fases:

8.1 Erros Léxicos

```
ERRO LEXICO: caractere nao reconhecido: '_' - LINHA: 9
```

8.2 Erros Sintáticos

```
ERRO SINTATICO: fim de arquivo inesperado, possivelmente falta
    fechar '}' - LINHA: 14
```

8.3 Erros Semânticos

```
ERRO SEMANTICO: variavel 'x' nao declarada - LINHA: 9
```

Cada erro indica:

- Número da linha onde ocorreu
- Descrição clara do problema
- Contexto (nome da variável, operação, etc.)

9 Arquitetura do Compilador

9.1 Fluxo de Compilação

1. **Leitura do arquivo fonte**: O programa principal lê o arquivo `.cm`
2. **Análise léxica**: Flex tokeniza o código
3. **Análise sintática**: Bison constrói a AST
4. **Análise semântica**: Verificações durante a construção da AST

5. **Visualização**: Geração do arquivo DOT para GraphViz
6. **Geração de código**: Produção do código intermediário TAC
7. **Saída**: AST, tabela de símbolos, código TAC ou mensagens de erro

9.2 Módulos do Compilador

9.2.1 cminusLex.l (Analisador Léxico)

- Define padrões regex para tokens
- Trata comentários e espaços em branco
- Retorna tokens para o parser
- Detecta caracteres inválidos

9.2.2 cminusSintSem.y (Parser + Semântica + AST)

- Define a gramática da linguagem
- Constrói a AST durante o parsing
- Implementa a tabela de símbolos
- Realiza verificações semânticas
- Gerencia escopos
- Gera mensagens de erro
- Gera visualização GraphViz

9.2.3 parser_context.h/c (Contexto do Parser)

- Define estrutura ParserContext
- Funções para criar/destruir contexto
- Isola estado entre compilações
- Facilita testes e reutilização

9.2.4 code_generator.h/c (Gerador de Código TAC)

- Implementa geração de código de três endereços
- Define funções para expressões e comandos
- Gerencia temporários e labels
- Interface limpa através de `codeGen()`

9.3 Função Main

A função main do compilador gerencia todo o processo de compilação:

```c
int main(int argc, char **argv) {
    /* 1. Abre arquivo de entrada */
    if (argc > 1) {
        yyin = fopen(argv[1], "r");
        if (!yyin) {
            fprintf(stderr, "Erro: nao foi possivel abrir '%s'\n",
                    argv[1]);
            return 1;
        }
    } else {
        yyin = stdin;
    }

    /* 2. Cria contexto do parser */
    ParserContext *ctx = parser_context_create();
    if (!ctx) {
        fprintf(stderr, "Erro: falha ao criar contexto\n");
        return 1;
    }

    /* 3. Cria escopo global */
    enter_scope_ctx(ctx);

    /* 4. Adiciona funcoes built-in */
    insert_symbol_ctx(ctx, "input", TYPE_INT, KIND_FUNC, 0);
    insert_symbol_ctx(ctx, "output", TYPE_VOID, KIND_FUNC, 0);

    /* 5. Executa parser */
    yyparse(ctx);
```

```

    /* 6. Exibe tabela de simbolos */
    ExibirTabelaSimbolos_ctx(ctx);

    /* 7. Se sem erros, gera visualizacoes da AST */
    if (!ctx->has_errors && ctx->ast_root) {
        printf("\n");
        printf("=============================================\n");
        printf("ARVORE SINTATICA ABSTRATA (AST)\n");
        printf("=============================================\n");
        /* printTree(ctx->ast_root, 0);
        printf("=============================================\n"); */

        /* Gera arquivo DOT para GraphViz */
        printTreeDOT(ctx->ast_root, "ast.dot");

        /* Gera codigo intermediario (tres enderecos) */
        codeGen(ctx->ast_root);
    }

    /* 8. Libera recursos */
    free_all_scopes_ctx(ctx);
    int return_code = ctx->has_errors ? 1 : 0;
    parser_context_destroy(ctx);

    if (yyin != stdin) fclose(yyin);
    return return_code;
}
```

Listing 30: Funcao main do compilador

9.4 Estruturas de Dados Principais

- **Simbolo**: Entrada na tabela de símbolos
- **Escopo**: Nível de escopo com lista de símbolos
- **TreeNode**: Nó da AST
- **Identifier**: Informações de identificador em nó

10 Exemplos de Uso

10.1 Programa Correto

```
int x[10];

int minloc(int a[], int low, int high) {
    int i; int x; int k;
    k = low;
    x = a[low];
    i = low + 1;
    while (i < high) {
        if (a[i] < x) {
            x = a[i];
            k = i;
        }
        i = i + 1;
    }
    return k;
}

void sort(int a[], int low, int high) {
    int i; int k;
    i = low;
    while (i < high - 1) {
        int t;
        k = minloc(a, i, high);
        t = a[k];
        a[k] = a[i];
        a[i] = t;
        i = i + 1;
    }
}

void main(void) {
    int i;
    i = 0;
    while (i < 10) {
        x[i] = input();
        i = i + 1;
    }
    sort(x, 0, 10);
```

```
    i = 0;
    while (i < 10) {
        output(x[i]);
        i = i + 1;
    }
}
```

Listing 31: sort.cm - Programa de ordenação

Saída do compilador:

```
Analise sintatica concluida com sucesso!
Analise semantica: Nenhum erro encontrado.

-------------------------------------------------
|   TABELA DE SIMBOLOS DO COMPILADOR C-MINUS    |
-------------------------------------------------

NOME   | TIPO | CATEGORIA | ESCOPO| TAMANHO   | PARAMS | LINHA
-------------------------------------------------------------
main     | void        | FUNC      | 0    | -         | 0       | 31
sort     | void        | FUNC      | 0    | -         | 3       | 18
minloc   | int         | FUNC      | 0    | -         | 3       | 3
x        | int[]       | ARRAY     | 0    | 10        | -       | 1
output   | void        | FUNC      | 0    | -         | 0       | 0
input    | int         | FUNC      | 0    | -         | 0       | 0
k        | int         | VAR       | 1    | -         | -       | 4
x        | int         | VAR       | 1    | -         | -       | 4
i        | int         | VAR       | 1    | -         | -       | 4
high     | int         | VAR       | 1    | -         | -       | 3
low      | int         | VAR       | 1    | -         | -       | 3
a        | int[]       | ARRAY     | 1    | 0         | -       | 3
k        | int         | VAR       | 4    | -         | -       | 19
i        | int         | VAR       | 4    | -         | -       | 19
high     | int         | VAR       | 4    | -         | -       | 18
low      | int         | VAR       | 4    | -         | -       | 18
a        | int[]       | ARRAY     | 4    | 0         | -       | 18
t        | int         | VAR       | 5    | -         | -       | 22
i        | int         | VAR       | 6    | -         | -       | 32


============================================================
          ARVORE SINTATICA ABSTRATA (AST)
```

```
==============================================================

Arquivo GraphViz gerado: ast.dot
Para gerar a imagem, execute:
   dot -Tpng ast.dot -o ast.png
   dot -Tsvg ast.dot -o ast.svg (recomendado)


*** CODIGO INTERMEDIARIO (3 ENDERECOS) ***

array x[10]

func minloc:
param a
param low
param high
k = low
t0 = a[low]
x = t0
t1 = 1
t2 = low + t1
i = t2
L0:
t3 = i < high
if_false t3 goto L1
t4 = a[i]
t5 = t4 < x
if_false t5 goto L2
t6 = a[i]
x = t6
k = i
L2:
t7 = 1
t8 = i + t7
i = t8
goto L0
L1:
return k
endfunc

func sort:
```

```
param a
param low
param high
i = low
L4:
t9 = 1
t10 = high - t9
t11 = i < t10
if_false t11 goto L5
param a
param i
param high
t12 = call minloc, 3
k = t12
t13 = a[k]
t = t13
t14 = a[i]
a[k] = t14
a[i] = t
t15 = 1
t16 = i + t15
i = t16
goto L4
L5:
endfunc

func main:
t17 = 0
i = t17
L6:
t18 = 10
t19 = i < t18
if_false t19 goto L7
t20 = call input, 0
x[i] = t20
t21 = 1
t22 = i + t21
i = t22
goto L6
L7:
param x
```

```
t23 = 0
param t23
t24 = 10
param t24
call sort, 3
t25 = 0
i = t25
L8:
t26 = 10
t27 = i < t26
if_false t27 goto L9
t28 = x[i]
param t28
call output, 1
t29 = 1
t30 = i + t29
i = t30
goto L8
L9:
endfunc

*****************************************
```

AST gerada em Graphviz

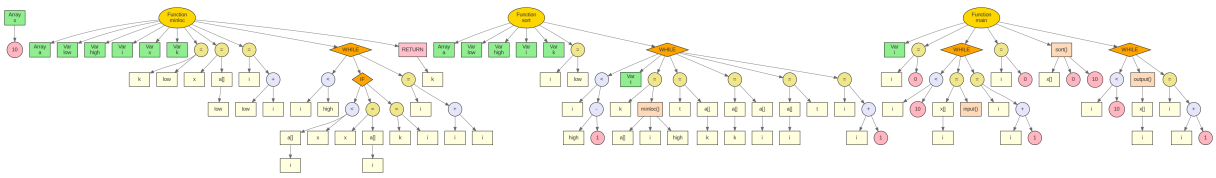

Figura 2: Visualização da árvore do exemplo em Graphviz

10.2 Programa com Erros

```
int main(int x) {
    int x;          // OK: x em escopo diferente
    void y;         // ERRO: variavel void
    int arr[10];

    y = 5;          // ERRO: atribuicao a void
    arr = 10;       // ERRO: atribuir a array
```

```
8
9       z = x + 5;        // ERRO: z nao declarada
10
11      x = y;            // ERRO: tipos incompativeis
12
13      if (y) { }        // ERRO: condicao nao e int
14  }
15
16  void soma(void a, void b) {  // ERRO: parametros void
17      return a + b;            // ERRO: a e b nao declarados
18  }
```

Listing 32: test_errors.cm - Programa com erros

Saída do compilador:

```
1  ERRO SEMANTICO: identificador 'x' ja declarado neste escopo -
      LINHA: 2
2  ERRO SEMANTICO: tipos incompativeis na atribuicao - LINHA: 6
3  ERRO SEMANTICO: nao e possivel atribuir a array completo -
      LINHA: 7
4  ERRO SEMANTICO: variavel 'z' nao declarada - LINHA: 9
5  ERRO SEMANTICO: tipos incompativeis na atribuicao - LINHA: 11
6  ERRO SEMANTICO: condicao do IF deve ser inteira - LINHA: 14
7  ERRO SEMANTICO: parametro 'a' nao pode ser void - LINHA: 16
8  ERRO SEMANTICO: parametro 'b' nao pode ser void - LINHA: 16
9  ERRO SEMANTICO: variavel 'a' nao declarada - LINHA: 17
10 ERRO SEMANTICO: variavel 'b' nao declarada - LINHA: 17
11 Analise sintatica concluida com sucesso!
12
13 ------------------------------------------------
14 |   TABELA DE SIMBOLOS DO COMPILADOR C-MINUS   |
15 ------------------------------------------------
16
17 NOME   | TIPO | CATEGORIA | ESCOPO| TAMANHO   | PARAMS | LINHA
18 ---------------------------------------------------------------
19 soma   | void | FUNC      | 0   | -   | 0       | 16
20 main   | int  | FUNC      | 0   | -   | 1       | 1
21 output | void | FUNC      | 0   | -   | 0       | 0
22 input  | int  | FUNC      | 0   | -   | 0       | 0
23 arr    | int[] | ARRAY    | 1   | 10  | -       | 4
24 y      | void | VAR       | 1   | -   | -       | 3
25 x      | int  | VAR       | 1   | -   | -       | 1
```

Neste exemplo a visualização da árvore em Graphviz e o código intermediário não são gerados.

11 Compilação e Execução

11.1 Ferramentas Necessárias

- **Flex**: Gerador de analisadores léxicos
- **Bison**: Gerador de parsers
- **GCC**: Compilador C

11.2 Processo de Compilação

```
# 1. Gerar analisador lexico
flex cminusLex.l

# 2. Gerar analisador sintatico
bison -d cminusSintSem.y -o cminus.tab.c

# 3. Compilar o compilador
gcc -o cminus cminus.tab.c lex.yy.c -Wall

# 4. Executar com arquivo de teste
.\cminus test.cm
```

11.3 Makefile

Um Makefile está disponível para automatizar a compilação, é possível conferi-lo no repositório do Github.

12 Conclusão

O compilador C-Minus desenvolvido cumpre com os requisitos estabelecidos para o projeto, apresentando uma implementação que integra todas as etapas do processo de compilação. O sistema realiza a análise léxica e sintática completa, garantindo o reconhecimento de todos os *tokens* e a validação da gramática integral da linguagem. Durante essas fases, o compilador constrói uma Árvore Sintática Abstrata (AST) com estrutura hierárquica detalhada, permitindo a inspeção do código tanto via terminal quanto por representações gráficas em `GraphViz`.

A consistência do código-fonte é assegurada por um motor semântico que gerencia escopos aninhados por meio de uma tabela de símbolos eficiente, validando declarações e tipos, inclusive para estruturas complexas como *arrays* e funções. O sistema possui um mecanismo de tratamento de erros capaz de identificar e reportar falhas em todas as etapas, interrompendo o fluxo quando necessário. Por fim, o ciclo de compilação culmina na geração de código intermediário no formato de três endereços (TAC), fornecendo uma base sólida para a tradução para arquiteturas de destino. Assim, para futuras disciplinas como Laboratório de Compiladores o compilador implementado permite futuras melhorias para síntese de código em formato esperado do processador.

12.1 Separação do Trabalho

Todos os integrantes participaram ativamente da implementação de todos os passos do compilador, mas cada um deu foco na implementação de algumas partes principais compilador. O aluno Jonas Lucas Durão ficou responsável pela montagem da analisador Sintático com a implementação da AST e da implementação das lógicas na gramática. O aluno Gabriel Leal Bonavina ficou responsável pelo analisador Léxico, tabela de símbolos e gerador de código. O aluno Marcelo Fukuya Ohno ficou responsável pelo analisador semântico e a adaptação da AST para o graphviz, também de melhoramento das mensagens de erro. Depois de finalizado o projeto com uma implementação inicial, foram feitos vários testes por todos do grupo para fazer Debug e refatoração de algumas partes que apresentaram problemas ou não estavam cumprindo com os requisitos pedidos pelo professor da disciplina.

13 Referências

1. AHO, Alfred V. et al. *Compilers: Principles, Techniques, and Tools.* 2nd ed. Pearson, 2006.

2. LEVINE, John. *flex & bison.* O'Reilly Media, 2009.

3. Material de apoio da disciplina - *Trabalho_Final_Compiladores.pdf*

4. Material de apoio - *AAS_Material_de_Apoio_Compiladores.pdf*